

IMPLEMENTATION SPECIFICATION

NexusAI — Enterprise AI Operations Platform

Complete Task List — LLM-Ready Implementation Specs

53 tasks • 7 phases • 45 days • Governed AI gateway, RAG, evaluation, observability and agent workflows

How to use this document

- Each task is a self-contained LLM prompt. Feed the **GOAL + SPEC + DONE WHEN** section directly to an LLM to generate the implementation. The SPEC provides enough context that the LLM does not need further clarification.
- Work tasks in phase order (Phase 0 → 6) and within each phase respect the **Depends** field. A task with no dependencies can be started immediately once its phase begins.
- Track** tells you which hat you are wearing: **PLAT** = platform/backend services, **DATA** = data pipelines, retrieval and evaluation, **UI** = frontend console, **OPS** = infrastructure and observability. On a solo build these are sequencing hints, not people.
- The **TABLES** row shows exactly which database tables the task reads from or writes to. Cross-reference with the Database Schema Specification for column details.
- Every task's **DONE WHEN** section is the acceptance test. A task is not complete until every statement in that section is verifiable.

Task summary by phase

Phase	Tasks	PLAT	DATA	UI	OPS	Modules covered
0 — Foundation & Environment	8	4	2	0	2	Repository, Local stack, Database, Identity
1 — AI Gateway	9	7	1	1	0	Gateway, Routing, Cost, Quotas, Console shell
2 — Knowledge Hub	9	4	4	1	0	Ingestion, Retrieval, RAG, Citations
3 — Observability & Cost	7	0	2	1	4	Tracing, Metrics, Dashboards, Alerts
4 — Evaluation Lab	6	2	3	1	0	Prompts, Datasets, Scorers, Experiments
5 — Agent Studio	7	5	1	1	0	Orchestration, Tools, Approvals, Reference agents
6 — Governance & Release	7	4	1	1	1	PII, Policy, Audit, Hardening, Portfolio
Total	53	26	14	6	7	

Task index with dependencies

Task ID	Title	Phase	Track	Day	Depends on
FOUND-001	Repository, tooling and project scaffolding	P0	PLAT	D1	—
FOUND-002	Docker Compose local stack	P0	OPS	D1–2	FOUND-001
FOUND-003	Configuration and secrets management	P0	PLAT	D2	FOUND-001
INFRA-001	PostgreSQL 16 + pgvector and Alembic setup	P0	DATA	D2	FOUND-002
INFRA-002	Migrations 001–003: identity, gateway, cost	P0	DATA	D2–3	INFRA-001
INFRA-003	Migrations 004–010: knowledge, agents, evaluation, observability, governance	P0	DATA	D3–4	INFRA-002
AUTH-001	Authentication — OIDC/SSO login, sessions, tokens	P0	PLAT	D3–4	INFRA-002, FOUND-003
AUTH-002	RBAC — roles, permissions, workspace scoping	P0	PLAT	D4–5	AUTH-001
GW-001	Provider abstraction layer	P1	PLAT	D5–7	FOUND-003, INFRA-002
GW-002	POST /v1/chat — unified completion endpoint	P1	PLAT	D7–8	GW-001, AUTH-002
GW-003	Model registry and routing policy engine	P1	PLAT	D8–10	GW-002
GW-004	Fallback and retry orchestration	P1	PLAT	D10–11	GW-003
GW-005	Streaming responses over SSE	P1	PLAT	D9–10	GW-002
GW-006	Token accounting and cost	P1	DATA	D10–11	GW-002

	calculation				
GW-007	Rate limiting and workspace quotas	P1	PLAT	D11–12	GW-002
GW-008	Request and response logging with redaction hooks	P1	PLAT	D11–12	GW-002, INFRA-003
UI-001	Console shell — layout, navigation, guarded routes	P1	UI	D12–14	AUTH-001
KB-001	Object storage and document upload API	P2	PLAT	D14–15	FOUND-002, AUTH-002
KB-002	Ingestion worker — parsing and chunking	P2	DATA	D15–17	KB-001, INFRA-003
KB-003	Embedding pipeline and pgvector indexing	P2	DATA	D17–18	KB-002, GW-001
KB-004	Hybrid retrieval — BM25 + vector with rank fusion	P2	DATA	D18–19	KB-003
KB-005	Access-controlled retrieval filter	P2	PLAT	D19	KB-004, AUTH-002
KB-006	RAG answer endpoint with citations	P2	PLAT	D19–20	KB-005, GW-002
KB-007	Freshness checks and re-index scheduling	P2	DATA	D20–21	KB-003
KB-008	Answer feedback capture	P2	PLAT	D21	KB-006
UI-002	Knowledge Hub UI — collections, upload, cited chat	P2	UI	D20–22	KB-006, UI-001
OBS-001	OpenTelemetry instrumentation — traces and spans	P3	OPS	D20–21	GW-002, FOUND-002
OBS-002	GenAI semantic conventions for LLM spans	P3	OPS	D21–22	OBS-001
OBS-003	Trace persistence and query API	P3	DATA	D22–23	OBS-001, INFRA-003
OBS-004	Metric rollups — hourly and daily aggregation	P3	DATA	D23–24	OBS-003, GW-006
OBS-005	Prometheus metrics and Grafana dashboards	P3	OPS	D24–25	OBS-001, FOUND-002
OBS-006	Alert rules and failure budget tracking	P3	OPS	D25–26	OBS-004
UI-003	Observability dashboard — cost, latency, usage, failures	P3	UI	D25–27	OBS-004, UI-001
EVAL-001	Prompt registry with versioning	P4	PLAT	D26–27	INFRA-003, AUTH-002
EVAL-002	Evaluation datasets and item management	P4	DATA	D27–28	EVAL-001
EVAL-003	Scorers — exact match, LLM-as-judge, retrieval metrics	P4	DATA	D28–30	EVAL-002, GW-002
EVAL-004	Evaluation run executor with MLflow tracking	P4	DATA	D30–31	EVAL-003, OBS-003
EVAL-005	Model and prompt comparison API	P4	PLAT	D31–32	EVAL-004
UI-004	Evaluation Lab UI — runs, comparisons, score breakdowns	P4	UI	D31–33	EVAL-005, UI-001
AGENT-001	LangGraph runtime integration and checkpointing	P5	PLAT	D32–34	GW-002, INFRA-003
AGENT-002	Tool registry and permission grants	P5	PLAT	D34–35	AGENT-001, AUTH-002
AGENT-003	Agent definition and versioning API	P5	PLAT	D35–36	AGENT-002
AGENT-004	Human-in-the-loop approval gates	P5	PLAT	D35–36	AGENT-001, INFRA-003
AGENT-005	Reference agent — document review with retrieval	P5	PLAT	D36–37	AGENT-003, KB-006
AGENT-006	Reference agent — data quality triage with approval	P5	DATA	D37–38	AGENT-004, AGENT-003

UI-005	Agent Studio UI — builder, run traces, approval queue	P5	UI	D37-39	AGENT-004, UI-001
GOV-001	PII detection and redaction pipeline	P6	PLAT	D38-39	GW-008, INFRA-003
GOV-002	Policy engine — model allow-lists, tool blocks, data rules	P6	PLAT	D39-40	GW-003, AGENT-002
GOV-003	Immutable audit log and compliance export	P6	DATA	D40-41	INFRA-003, AUTH-002
GOV-004	Security hardening	P6	PLAT	D41-42	GW-007, KB-005, AGENT-002
UI-006	Governance Center and Admin Console	P6	UI	D41-43	GOV-002, GOV-003, UI-001
SHIP-001	Seed data, demo script and end-to-end walkthrough	P6	PLAT	D43-44	UI-006, AGENT-006, EVAL-005, UI-003
SHIP-002	Documentation, architecture diagram and write-up	P6	OPS	D44-45	SHIP-001

Two tasks have no dependencies and can both start on day one: FOUND-001 and, once the repository exists, FOUND-002 and FOUND-003 fork in parallel.

Phase 0 — Foundation & Environment

8 tasks · D1–D5

FOUND-001 · Repository, tooling and project scaffolding

Track PLAT · Day D1 · Depends —

GOAL — A single repository with the backend service, frontend app, shared packages and a working developer loop.

SPEC

/services/api/	FastAPI app – routers, services, models, workers
/services/worker/	Celery workers – ingestion, rollups, eval runs
/apps/console/	Next.js 14 app router + TailwindCSS
/packages/schemas/	Pydantic models shared across api and worker
/packages/sdk-python/	Thin client for the gateway (used by agents and evals)
/infra/	Docker Compose, Grafana dashboards, OTel collector config
/migrations/	Alembic versions

- Python 3.12, `uv` or Poetry for dependency management, `ruff + mypy --strict` on the API package.
- Node 20, pnpm workspaces for the console.
- `Makefile` targets: `make dev`, `make test`, `make lint`, `make migrate`, `make seed`.
- Pre-commit hooks: `ruff`, `mypy`, `prettier`, secret scanning (`gitleaks`).
- GitHub Actions `ci.yml`: `lint`, `type-check`, `unit tests`, `build both images` on every PR.

DONE WHEN — `make dev` starts API and console locally. `make lint` and `make test` pass on a clean checkout. CI is green on the first PR.

FOUND-002 · Docker Compose local stack

Track OPS · Day D1–2 · Depends FOUND-001

GOAL — Every backing service the platform needs runs locally with one command.

SPEC - `postgres:16` with the `pgvector` extension image, port 5432, named volume. - `redis:7` — Celery broker and rate-limit buckets. - `minio` — S3-compatible object storage for uploaded documents, bucket `nexus-documents` created on start. - `mlflow` — tracking server backed by the same Postgres, artifacts to MinIO. - `otel-collector` — receives OTLP, exports to Tempo/Jaeger and Prometheus. - `prometheus + grafana` — Grafana provisioned with the datasource and an empty dashboard folder. - Healthchecks on every service; `depends_on: condition: service_healthy` so `make dev` blocks until ready. - `.env.example` documenting every variable with a safe default.

DONE WHEN — `docker compose up` brings all services healthy. MLflow UI loads. Grafana shows the Prometheus datasource. `psql` can create a vector column.

FOUND-003 · Configuration and secrets management

Track PLAT · Day D2 · Depends FOUND-001

GOAL — One typed settings object; no secret ever read directly from `os.environ` in application code.

SPEC - `pydantic-settings Settings` class with nested sections: `database`, `redis`, `storage`, `auth`, `providers`, `telemetry`. - Provider credentials modelled as a dict of `ProviderCredential` keyed by provider slug, so adding a provider is config, not code. - Backends behind one interface: environment variables locally, and a `SecretResolver` protocol with an env implementation plus a stub for a managed secret store. - Secrets are never logged. Add a logging filter that redacts any value matching a registered secret. - `GET /health` returns build SHA, environment name and the list of configured providers — never the credentials.

DONE WHEN — Settings fail fast at startup with a clear message when a required value is missing. No secret appears in logs at DEBUG level. Adding a new provider needs no code change in the settings module.

INFRA-001 · PostgreSQL 16 + pgvector and Alembic setup

Track DATA · Day D2 · Depends FOUND-002

GOAL — Database connection, migration tooling and the extensions the platform depends on.

SPEC - SQLAlchemy 2.0 async engine, session factory with `expire_on_commit=False`, connection pool sized for the worker count. - Alembic configured with async support and `compare_type=True` so column type changes are caught. - Baseline migration `000_extensions: CREATE EXTENSION IF NOT EXISTS vector; pgcrypto; pg_trgm;` - `vector` — embeddings, `pgcrypto` — UUID generation and column encryption, `pg_trgm` — keyword search in hybrid retrieval. - Naming convention set on `MetaData` so constraint names are deterministic across migrations. - `make migrate` runs `alembic upgrade head`; `make revision m="..."` autogenerates.

DONE WHEN — `alembic upgrade head` succeeds on an empty database. All three extensions are installed. Autogenerate produces a clean empty diff immediately after upgrade.

TABLES — all (host)

INFRA-002 • Migrations 001-003: identity, gateway, cost

Track DATA • Day D2-3 • Depends INFRA-001

GOAL — The tables needed before any request can be authenticated, routed or costed.

SPEC - 001 — Identity & access: `users`, `workspaces`, `workspace_memberships`, `roles`, `role_assignments`, `api_keys`, `user_sessions` - `users.email` UNIQUE; `api_keys.key_hash` UNIQUE; `workspace_memberships (workspace_id, user_id)` UNIQUE. - **002 — Gateway & routing:** `providers`, `models`, `model_deployments`, `routing_policies`, `routing_rules`, `gateway_requests`, `gateway_messages`, `fallback_events` - Seed `providers` with openai, anthropic, azure_openai and a `mock` provider used by tests and offline development. - Seed `models` with at least one cheap and one strong model per provider, including context window and modality flags. - **003 — Cost & quotas:** `pricing_rates`, `cost_entries`, `budgets`, `quota_windows`, `rate_limit_buckets` - `pricing_rates` is effective-dated: `(model_id, effective_from)` with no overlaps, so historical costs stay correct when prices change. - All status and type columns are `VARCHAR` + `CHECK`, never native Postgres `ENUM`.

DONE WHEN — Migrations apply cleanly. Seeds create the mock provider and at least four models. Foreign keys enforce referential integrity in both directions.

TABLES — `users`, `workspaces`, `workspace_memberships`, `roles`, `role_assignments`, `api_keys`, `user_sessions`, `providers`, `models`, `model_deployments`, `routing_policies`, `routing_rules`, `gateway_requests`, `gateway_messages`, `fallback_events`, `pricing_rates`, `cost_entries`, `budgets`, `quota_windows`, `rate_limit_buckets`

INFRA-003 • Migrations 004-010: knowledge, agents, evaluation, observability, governance

Track DATA • Day D3-4 • Depends INFRA-002

GOAL — Every remaining table, plus triggers, vector indexes, append-only enforcement and partitioning.

SPEC - 004 — Knowledge Hub: `collections`, `collection_grants`, `documents`, `document_versions`, `chunks`, `chunk_embeddings`, `ingestion_jobs`, `retrieval_queries`, `retrieval_citations` - `chunk_embeddings.embedding vector(1536)`; HNSW index `hnsw (embedding vector_cosine_ops)`. - GIN trigram index on `chunks.content` for the keyword half of hybrid retrieval. - **005 — Agents:** `agent_definitions`, `agent_versions`, `tools`, `agent_tool_grants`, `agent_runs`, `agent_steps`, `approval_requests` - **006 — Evaluation:** `prompt_templates`, `prompt_versions`, `eval_datasets`, `eval_items`, `scorers`, `eval_runs`, `eval_results` - **007 — Observability:** `traces`, `spans`, `span_events`, `metric_rollups`, `alert_rules` - **008 — Governance:** `policies`, `policy_violations`, `pii_detections`, `redaction_rules`, `feature_flags`, `data_retention_schedules` - **009 — Triggers:** shared `set_updated_at()` applied to every table carrying `updated_at`; `log_audit()` writing old/new JSONB to `audit_log` on `users`, `api_keys`, `routing_policies`, `policies`, `agent_tool_grants`, `collection_grants`. - **010 — Immutability and partitioning:** `audit_log` created as a range-partitioned table by month with partitions through the next four quarters; RLS policies blocking UPDATE and DELETE on `audit_log`, `cost_entries`, `policy_violations`, `agent_steps`, `eval_results`.

DONE WHEN — All migrations apply. A `DELETE` on `audit_log` is rejected. Vector similarity search returns rows against a seeded embedding. `updated_at` changes on UPDATE without application code touching it.

TABLES — `collections`, `collection_grants`, `documents`, `document_versions`, `chunks`, `chunk_embeddings`, `ingestion_jobs`, `retrieval_queries`, `retrieval_citations`, `agent_definitions`, `agent_versions`, `tools`, `agent_tool_grants`, `agent_runs`, `agent_steps`, `approval_requests`, `prompt_templates`, `prompt_versions`, `eval_datasets`, `eval_items`, `scorers`, `eval_runs`, `eval_results`, `traces`, `spans`, `span_events`, `metric_rollups`, `alert_rules`, `audit_log`, `policies`, `policy_violations`, `pii_detections`, `redaction_rules`, `feature_flags`, `data_retention_schedules`

AUTH-001 • Authentication — OIDC/SSO login, sessions, tokens

Track PLAT • Day D3-4 • Depends INFRA-002, FOUND-003

GOAL — Humans log in through an identity provider; services authenticate with API keys. Both resolve to the same principal object.

SPEC - OIDC authorization-code flow with PKCE against a configurable issuer. Local development uses a self-hosted Keycloak or Dex container; the same code path works with Entra ID or Okta. - On first successful login, provision the `users` row and attach them to a default workspace with the `viewer` role. - Session: opaque token, SHA-256 stored in `user_sessions.token_hash`, 8-hour expiry, sliding refresh capped at 24 hours. Store in an httpOnly, SameSite=Lax cookie. - API keys: `nx_live_ / nx_test_` prefix, 32 random bytes, SHA-256 hash stored, full key shown exactly once at creation. Scoped to one workspace. - One FastAPI dependency `get_principal()` returns a `Principal` with `type (user | service)`, `user_id`, `workspace_id`, `roles` and `scopes` — regardless of which credential was presented. - Revocation: setting `user_sessions.revoked_at` or `api_keys.revoked_at` takes effect on the next request, checked against a Redis cache with a 30-second TTL.

DONE WHEN — OIDC login creates a user and a session. An API key authenticates a request. A revoked key returns 401 within 30 seconds. `get_principal()` is the only place either credential type is read.

TABLES — `users`, `workspaces`, `workspace_memberships`, `user_sessions`, `api_keys`

AUTH-002 • RBAC — roles, permissions, workspace scoping

Track PLAT • Day D4-5 • Depends AUTH-001

GOAL — Every query is scoped to a workspace, and every mutating endpoint checks a named permission.

SPEC - Roles: `owner`, `admin`, `engineer`, `analyst`, `viewer`. Permissions are strings namespaced by module, for example `gateway.route.write`, `knowledge.collection.read`, `agents.tool.grant`, `governance.policy.write`. - `roles.permissions` is JSONB so a role can be extended without a migration. `role_assignments` binds a user to a role within a workspace. - `require(permission)` FastAPI dependency raising 403 with the missing permission named in the response body — a generic “forbidden” is useless when you are debugging a policy. - Workspace scoping enforced in the repository layer, not the router: every repository method takes `workspace_id` and includes it in the WHERE clause. Write a test that reflects over the repositories and fails if any public method is missing the parameter. - Cross-workspace access is impossible by construction, not by convention.

DONE WHEN — A viewer gets 403 on a write endpoint with the permission name in the body. A user in workspace A cannot read a record in workspace B even with a valid ID. The reflection test passes.

TABLES — `roles`, `role_assignments`, `workspace_memberships`

Phase 1 — AI Gateway

9 tasks · D5-D14 · The front door for every AI request

GW-001 · Provider abstraction layer

Track PLAT · Day D5-7 · Depends FOUND-003, INFRA-002

GOAL — One internal request shape that maps onto OpenAI, Anthropic, Azure OpenAI and a mock provider, so routing has something uniform to route.

SPEC - `LLMProvider` protocol: `complete(request) -> CompletionResult`, `stream(request) -> AsyncIterator[Chunk]`, `embed(texts) -> list[Vector]`, `health() -> ProviderHealth`. - Internal `CompletionRequest`: messages, model slug, temperature, max_tokens, stop, tools, response_format, metadata. Provider adapters translate both directions — no provider-specific field leaks into the internal type. - `CompletionResult` normalises: text, tool calls, finish reason, `usage.input_tokens`, `usage.output_tokens`, provider request ID, latency. - Error normalisation is the important part. Map every provider's failures into: `RateLimitError`, `ProviderTimeoutError`, `ContextLengthError`, `ContentFilterError`, `AuthError`, `ProviderUnavailableError`. Routing and fallback logic must never branch on a provider's raw error string. - `MockProvider` returns deterministic responses keyed by a hash of the prompt, and can be told to fail with any of the above errors — this is what makes fallback testable without spending money. - Async HTTP client with per-provider timeout and connection pool.

DONE WHEN — The same `CompletionRequest` returns a normalised result from all four adapters. Each provider error maps to the correct exception type, verified by a test per error class. `MockProvider` needs no network.

TABLES — providers, models, model_deployments

GW-002 · `POST /v1/chat` — unified completion endpoint

Track PLAT · Day D7-8 · Depends GW-001, AUTH-002

GOAL — The single endpoint every application in the organisation calls instead of calling a model provider directly.

SPEC - `POST /v1/chat` body: `{ messages, model?, policy?, max_tokens?, temperature?, tools?, metadata?, stream? }` - Exactly one of `model` (pin a specific model) or `policy` (let the router decide). Reject both or neither with 400. - Pipeline, in order: authenticate → resolve workspace → check quota → apply pre-request policy → route → call provider → record usage and cost → log request → return. - Every stage is a middleware-style component in an explicit list, so GW-007, GW-008, GOV-001 and GOV-002 slot in later without rewriting the handler. - Write `gateway_requests` on entry (status `in_flight`) and update on completion. Write one `gateway_messages` row per message with the content reference, not the content, when redaction is enabled. - Response includes `x-nexus-request-id`, the model actually used, and token counts — callers need to see when routing sent them somewhere different. - Idempotency: honour an `Idempotency-Key` header; replay the stored response for 24 hours.

DONE WHEN — A request with `model` pinned returns a completion. A request with an unknown model returns 404 naming the model. The request appears in `gateway_requests` with token counts. Replaying an idempotency key returns the identical response without calling the provider.

TABLES — `gateway_requests`, `gateway_messages`, models, workspaces

GW-003 · Model registry and routing policy engine

Track PLAT · Day D8-10 · Depends GW-002

GOAL — A named policy decides which model serves a request, from declarative rules rather than code.

SPEC - `routing_policies` is a named, versioned, workspace-scoped object. `routing_rules` are ordered conditions evaluated top to bottom; first match wins; the policy carries a default model for no match. - Rule conditions available: `metadata.task_type` equals/in, estimated input tokens above/below a threshold, `metadata.tier`, caller API key, time of day, and a boolean `requires_tools`. - Rule actions: route to a model, route to a model list in preference order, or reject with a reason. - Worked example, which should ship as the seeded default policy: - `task_type in (summarise, classify, extract)` and estimated tokens < 4000 → cheap model - `task_type in (legal_review, code_review, analysis)` → strong model - `requires_tools = true` → strong model - default → mid-tier model - Estimate input tokens with the provider's tokeniser where available, `len(text)/4` otherwise. The estimate is used for routing only; billing uses the real count from GW-006. - Policy changes are versioned. `gateway_requests` stores the `routing_policy_version_id` used, so a cost spike can be traced to the policy edit that caused it. - `POST /v1/routing-policies/{id}/simulate` replays the last N requests against a draft policy and reports what would have changed — test a policy before it is live.

DONE WHEN — A summarisation request routes to the cheap model and a legal-review request to the strong model, from the same endpoint with no code difference. The policy version is recorded on the request. Simulate returns a diff without calling any provider.

TABLES — `routing_policies`, `routing_rules`, models, `gateway_requests`

GW-004 · Fallback and retry orchestration

Track PLAT · Day D10-11 · Depends GW-003

GOAL — A provider outage degrades quality, not availability.

SPEC - Retry policy per normalised error class, not per provider: - `RateLimitError` → retry same model with exponential backoff and jitter, max 2 attempts, honour `Retry-After` when present. - `ProviderTimeoutError`, `ProviderUnavailableError` → immediately fall back to the next model in the policy's preference list. - `ContextLengthError` → fall back to a model with a larger context window if the policy offers one, otherwise fail fast. - `ContentFilterError`, `AuthError` → never retry, never fall back. Return immediately. - Circuit breaker per `(provider, model)`: open after 5 consecutive failures, half-open probe after 30 seconds, close after 2 successes. State in Redis so all API replicas share it. - Every fallback writes a `fallback_events` row with the from-model, to-model, trigger error class and attempt number. This table is what proves the gateway earned its place. - Total wall-clock budget per request, default 60 seconds. When the budget is exhausted, stop falling back and return the last error.

DONE WHEN — Forcing the mock provider to fail produces a successful response from the fallback model and one `fallback_events` row. Five consecutive failures open the circuit and subsequent requests skip that model entirely. An auth error returns in one attempt.

TABLES — `fallback_events`, `gateway_requests`, `models`

GW-005 • Streaming responses over SSE

Track PLAT • Day D9–10 • Depends GW-002

GOAL — Token-by-token streaming through the gateway, with accounting that still works.

SPEC - `stream: true` returns `text/event-stream`. Event types: `delta` (content), `tool_call`, `usage`, `error`, `done`. - Usage is only known at the end for most providers — emit a `usage` event before `done` and write `gateway_requests` and `cost_entries` from the stream's completion handler, not the request handler. - Client disconnect must not orphan the record: wrap the generator so that on `asyncio.CancelledError` the request is marked `cancelled` with partial token counts. - Fallback during streaming is only possible before the first `delta` is emitted. After that, fail the stream with an `error` event; do not silently switch models mid-answer.

DONE WHEN — A streamed request delivers deltas incrementally, ends with usage and done, and writes a complete cost entry. Disconnecting mid-stream leaves a `cancelled` record, not an `in_flight` one.

TABLES — `gateway_requests`, `cost_entries`

GW-006 • Token accounting and cost calculation

Track DATA • Day D10–11 • Depends GW-002

GOAL — Every request has a defensible cost attached to a workspace, a user and a model.

SPEC - `pricing_rates` holds input and output price per million tokens, per model, effective-dated. Cached in memory with a 5-minute TTL. - On completion, write one `cost_entries` row: request id, workspace, user or API key, model, input tokens, output tokens, cached tokens, the rate ID applied, and the computed cost. - Use `NUMERIC(14,8)` for cost. Per-request costs are fractions of a cent and float rounding accumulates visibly across a month. - Cost is computed against the rate effective at request time, so historical reports do not change when you update prices. - Separate cost lines for embeddings so RAG ingestion cost is visible independently of query cost — otherwise a large ingestion looks like a chat cost spike. - `GET /v1/costs?group_by=workspace|user|model|day&from=&to=` returns aggregates.

DONE WHEN — A completed request produces exactly one cost entry with the right rate. Changing a price creates a new effective-dated rate without altering past entries. Embedding cost is queryable separately from completion cost.

TABLES — `cost_entries`, `pricing_rates`, `gateway_requests`, `models`

GW-007 • Rate limiting and workspace quotas

Track PLAT • Day D11–12 • Depends GW-002

GOAL — No single team can exhaust a shared provider quota or a monthly budget.

SPEC - Two independent controls, both enforced before the provider call: - **Rate limit** — requests and tokens per minute, per API key and per workspace. Sliding-window counter in Redis (`rate_limit_buckets` mirrors state for the console). - **Budget** — monetary cap per workspace per calendar month, from `budgets`. Soft threshold at 80% emits a warning header and an alert; hard cap at 100% rejects with 402. - `quota_windows` stores usage per workspace per window for reporting and for surviving a Redis flush. - Rejections return 429 (rate) or 402 (budget) with `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset` and a body explaining which control fired and when it resets. A caller must be able to self-diagnose. - Enforcement is fail-closed on budget and fail-open on rate limiting: if Redis is unavailable, allow traffic but log loudly. Losing observability should not take down the platform, but an unbounded spend should.

DONE WHEN — Exceeding the per-minute limit returns 429 with correct headers. Exceeding the monthly budget returns 402. The 80% threshold fires exactly once per window. Redis being down does not block requests.

TABLES — `rate_limit_buckets`, `quota_windows`, `budgets`, `cost_entries`

GW-008 • Request and response logging with redaction hooks

Track PLAT • Day D11–12 • Depends GW-002, INFRA-003

GOAL — Full prompt and response history, stored in a way that a compliance team would accept.

SPEC - `gateway_messages` stores role, sequence, content, token count and a `content_ref` pointing at object storage when the content exceeds 8KB. - Three configurable retention modes per workspace, on `workspaces.logging_mode`: - `full` — store prompt and response verbatim - `redacted` — store content with PII spans replaced (GOV-001 fills this in; until then the hook is a no-op) - `metadata_only` — store token counts, model and timings, no content at all - The redaction hook is a registered pipeline stage now, so GOV-001 is a plug-in rather than a rewrite. - Retention: `data_retention_schedules` row per table; a nightly job deletes or anonymises message content past the window while keeping the request and cost rows forever. - Logs are queryable by request ID, workspace, user, model and time range, with the content field omitted unless the caller holds `gateway.content.read`.

DONE WHEN — Messages are stored and retrievable by request ID. Switching a workspace to `metadata_only` stops content being persisted. A user without `gateway.content.read` sees metadata but not prompts. The retention job removes content past the window and leaves cost history intact.

TABLES — `gateway_messages`, `gateway_requests`, `data_retention_schedules`

UI-001 · Console shell — layout, navigation, guarded routes

Track UI · Day D12-14 · Depends AUTH-001

GOAL — The application frame every later module renders inside.

SPEC

/apps/console/app/	
(auth)/login/	OIDC redirect and callback handling
(app)/layout.tsx	sidebar, workspace switcher, user menu
(app)/overview/	landing: today's requests, spend, error rate
(app)/gateway/	requests explorer, routing policies
(app)/knowledge/	collections and chat (UI-002)
(app)/observability/	cost, latency, usage (UI-003)
(app)/evaluation/	runs and comparisons (UI-004)
(app)/agents/	builder, runs, approvals (UI-005)
(app)/governance/	policies, audit, admin (UI-006)

- `middleware.ts` guards `(app)` routes, redirects to login, and preserves the intended destination.
- Workspace switcher writes to a cookie; every API call sends the active workspace.
- Shared components: `DataTable` with server-side pagination and filters, `StatCard`, `StatusPill`, `CostDisplay` (renders sub-cent values without rounding them to zero), `TraceTimeline`, `EmptyState`.
- Navigation items are permission-gated — a viewer does not see the Governance section at all.
- Data fetching with SWR, 30-second revalidation on dashboards, no polling on static pages.

DONE WHEN — Unauthenticated users land on login and return to their intended page after signing in. Switching workspaces changes the data on screen. Permission-gated nav items are hidden, not just disabled. Every route renders an intentional empty state rather than a blank panel.

Phase 2 — Knowledge Hub

9 tasks · D14–D22 · Company data becomes answerable, with citations

KB-001 · Object storage and document upload API

Track PLAT · Day D14–15 · Depends FOUND-002, AUTH-002

GOAL — Documents land in object storage with their metadata in Postgres, and permissions attached from the first moment.

SPEC - `POST /v1/collections` — create a collection: name, description, embedding model, chunking strategy, default access level. - `collection_grants` binds a collection to a role or a specific user with `read` or `manage`. Retrieval later filters on this table, so it has to exist before ingestion, not after. - `POST /v1/collections/{id}/documents` returns a presigned PUT URL (15-minute expiry) plus a `document_id`. The browser uploads directly to object storage; the API never proxies file bytes. - `POST /v1/documents/{id}/confirm` verifies the object exists, records size, MIME type and SHA-256, and enqueues an `ingestion_jobs` row. - Accept `pdf`, `docx`, `txt`, `md`, `html`, `csv`. Reject anything else with the accepted list in the error. - Re-uploading a document with a matching SHA-256 creates a new `document_versions` row rather than a duplicate document.

DONE WHEN — Upload via presigned URL succeeds and the object exists in MinIO. Confirm creates the document and queues an ingestion job. A user without a grant on the collection gets 403. Re-uploading identical content versions rather than duplicates.

TABLES — `collections`, `collection_grants`, `documents`, `document_versions`, `ingestion_jobs`

KB-002 · Ingestion worker — parsing and chunking

Track DATA · Day D15–17 · Depends KB-001, INFRA-003

GOAL — A document becomes chunks that carry enough context to be useful on their own.

SPEC - Celery worker consumes `ingestion_jobs`. States: `queued` → `parsing` → `chunking` → `embedding` → `indexed`, or `failed` with the stage and error recorded. - Parsers: `unstructured` or `pymupdf` for PDF, `python-docx` for Word, `markdown-it` for Markdown, `selectolax` for HTML, `pandas` for CSV. - Chunking strategies, selectable per collection: - `recursive` — 800 tokens with 120-token overlap, splitting on paragraph then sentence boundaries - `semantic` — split on embedding distance between adjacent sentences - `structural` — split on Markdown headings or PDF sections, keeping the heading path - Every chunk stores its provenance: document ID, version, page or section number, character offsets, and the heading path. Citations in KB-006 are only trustworthy if this is populated correctly. - Prepend the document title and heading path to each chunk's embedded text while keeping the raw text separate for display — this measurably improves retrieval on short chunks. - Idempotent by (`document_version_id`, `chunking_strategy`): re-running replaces the chunk set in one transaction rather than duplicating it. - Failures retry 3 times with backoff, then park the job as `failed` with the exception recorded for the console.

DONE WHEN — A 40-page PDF produces chunks with correct page numbers. Re-running ingestion produces the same chunk count, not double. A corrupt file fails cleanly with a readable reason instead of hanging the worker.

TABLES — `ingestion_jobs`, `documents`, `document_versions`, `chunks`

KB-003 · Embedding pipeline and pgvector indexing

Track DATA · Day D17–18 · Depends KB-002, GW-001

GOAL — Chunks become searchable vectors, embedded through the gateway so the cost is counted.

SPEC - Embeddings are requested through the gateway's provider layer, not by calling a provider SDK directly — that is what makes ingestion cost appear in `cost_entries` alongside everything else. - Batch 100 chunks per call, bounded concurrency, retry on rate limit with backoff. - `chunk_embeddings` stores `vector(1536)`, the model slug and the embedding version. Storing the model on the row is what lets you migrate embedding models without a full-table rebuild. - HNSW index with `m=16`, `ef_construction=64`; set `ef_search` per query for the recall/latency trade-off. - Normalise vectors on write and use cosine distance. - Re-embedding path: writing a new embedding version leaves the old rows in place until the new set is complete, then flips the collection's active version in one statement. No search downtime.

DONE WHEN — All chunks in a collection have embeddings. A similarity query returns sensible neighbours in under 100ms on 10,000 chunks. Embedding cost appears in `cost_entries` tagged as an embedding. A model switch completes without search returning empty results at any point.

TABLES — `chunk_embeddings`, `chunks`, `collections`, `cost_entries`

KB-004 · Hybrid retrieval — BM25 + vector with rank fusion

Track DATA · Day D18–19 · Depends KB-003

GOAL — Retrieval that handles both “what does the leave policy say” and an exact error code, because vector search alone is bad at the second.

SPEC - Keyword leg: Postgres full-text search with `ts_rank_cd`, plus trigram similarity for typo tolerance. - Vector leg: cosine similarity over `chunk_embeddings`. - Fuse with Reciprocal Rank Fusion: $score = \sum 1 / (k + rank_i)$, $k = 60$. RRF avoids having to normalise two incomparable score scales, which is where naive hybrid implementations go wrong. - Optional cross-encoder rerank on the top 30 fused results, behind a per-collection flag — it improves precision and costs latency, so it should be a decision, not a default. - Filters: collection, document, date range, and arbitrary metadata key/values. - Return top `k` (default 8) with fused score, both component ranks, and full chunk provenance. - `retrieval_queries` records the query text, filters, latency, result IDs and scores — the input to retrieval quality evaluation in EVAL-003.

DONE WHEN — An exact-phrase query outranks a semantically similar but wrong chunk. A conceptual query returns relevant chunks with no keyword overlap. Every query is recorded with its results. Disabling rerank measurably reduces latency.

KB-005 · Access-controlled retrieval filter

Track PLAT · Day D19 · Depends KB-004, AUTH-002

GOAL — Retrieval can never surface a chunk the caller is not entitled to read.

SPEC - Permission filtering happens **inside** the SQL query, joined against `collection_grants` — never as a post-filter on results. Post-filtering leaks through result counts and through the reranker. - Effective access resolves from: workspace membership, collection grants by role, explicit user grants, and document-level `sensitivity` labels (`public`, `internal`, `confidential`, `restricted`) matched against the principal's clearance. - Agents and service API keys are subject to the same filter as humans. An agent must never be able to retrieve more than the user it acts for — pass the delegating principal through to retrieval. - Every retrieval records the principal, the collections searched and the number of chunks excluded by permission. That count is an early warning that permissions are misconfigured. - Test with two workspaces holding documents containing an identical distinctive string, and assert each can only ever retrieve its own.

DONE WHEN — Cross-workspace retrieval returns zero results for the distinctive string. Removing a grant immediately removes those chunks from results. An agent acting for a viewer retrieves exactly what that viewer would.

TABLES — collection_grants, collections, documents, chunks, retrieval_queries

KB-006 · RAG answer endpoint with citations

Track PLAT · Day D19–20 · Depends KB-005, GW-002

GOAL — A grounded answer where every claim points back to a specific chunk of a specific document.

SPEC - `POST /v1/knowledge/ask` body: `{ question, collection_ids?, filters?, policy?, max_chunks? }` - Flow: retrieve (KB-005) → assemble context with numbered source markers → call the gateway (GW-002) with a grounding system prompt → parse citation markers → resolve to `retrieval_citations` rows. - The system prompt must instruct the model to answer only from the provided context, to cite with `[n]` markers, and to say plainly when the context does not contain the answer. An unanswerable question returning "the documents do not cover this" is a correct answer, and the evaluation set in EVAL-002 should contain several. - Response: `{ answer, citations: [{ marker, document_id, document_title, chunk_id, page, snippet, score }], model_used, tokens, request_id }` - Drop any citation marker the model emits that does not correspond to a supplied chunk, and record it — hallucinated citations are a quality signal worth measuring. - Context assembly respects the target model's context window with a safety margin, dropping the lowest-ranked chunks first and reporting how many were dropped.

DONE WHEN — An answer returns with citations resolving to real chunks with correct page numbers. A question the corpus does not cover returns an explicit no-answer rather than an invention. Invalid markers are stripped and counted. Oversized context is truncated by rank, never by truncating a chunk mid-sentence.

TABLES — retrieval_queries, retrieval_citations, gateway_requests, chunks

KB-007 · Freshness checks and re-index scheduling

Track DATA · Day D20–21 · Depends KB-003

GOAL — Answers stop being confidently wrong because a document changed six weeks ago.

SPEC - Nightly job flags documents whose source has changed (hash mismatch on re-fetch) or whose age exceeds the collection's `freshness_window_days`. - Stale documents surface in the console and are excluded from retrieval when the collection sets `exclude_stale = true`. - Answers include a freshness note when any cited document is past its window — the citation is still shown, but its age is visible. - Scheduled re-embedding when a collection's embedding model changes, using the zero-downtime path from KB-003. - Orphan cleanup: chunks whose document version is superseded and whose grace period has passed are deleted, freeing index space.

DONE WHEN — Modifying a source document flags it stale by the next run. A stale citation carries its age in the response. Re-index completes without search downtime. Orphaned chunks are removed.

TABLES — documents, document_versions, chunks, chunk_embeddings, ingestion_jobs, collections

KB-008 · Answer feedback capture

Track PLAT · Day D21 · Depends KB-006

GOAL — Bad answers become evaluation data instead of complaints.

SPEC - `POST /v1/knowledge/answers/{request_id}/feedback` body: `{ rating: helpful|unhelpful|incorrect, reason?, expected_answer?, bad_citation_ids? }` - Feedback attaches to the `retrieval_queries` row, so it carries the full context: question, retrieved chunks, scores, model used and answer. - Any answer marked `incorrect` with an `expected_answer` is eligible for promotion into an `eval_datasets` item — this is the loop that makes EVAL-002 populated with real cases rather than invented ones. - `bad_citation_ids` marks specific citations as not supporting the claim, which distinguishes a retrieval failure from a generation failure. Those are different bugs with different fixes. - Feedback is queryable filtered by collection and date, ordered by frequency of the underlying question.

DONE WHEN — Feedback persists against the query record. An incorrect answer with an expected answer appears in the eligible-for-dataset list. Retrieval failures and generation failures are separable in the feedback report.

TABLES — retrieval_queries, retrieval_citations, eval_items

Track UI · Day D20-22 · Depends KB-006, UI-001

GOAL — Upload documents, ask questions and inspect exactly where an answer came from.

SPEC - Collections list — name, document count, chunk count, embedding model, last indexed, freshness status. - **Collection detail** — drag-and-drop upload with per-file progress and live ingestion status (`parsing` → `chunking` → `embedding` → `indexed`), failures showing the stage and reason. - **Chat** — streamed answer with inline `[n]` citation chips. Clicking a chip opens a side panel with the chunk text, document title, page and relevance score, plus a link to the source document at that page. - **Retrieval inspector** — a collapsible panel under each answer showing every retrieved chunk with keyword rank, vector rank, fused score and whether it made it into the context. This is the single most useful debugging view in the whole product; give it real space. - Thumbs up/down plus a "wrong citation" action on each chip, wired to KB-008. - Access indicator on each collection showing who can read it, so a misconfigured grant is visible before it becomes an incident.

DONE WHEN — Upload shows live progress through every ingestion stage. Answers stream with working citation chips. The inspector shows both component ranks. Feedback submits from the chat. A collection the user cannot read does not appear in the list.

Phase 3 — Observability & Cost

7 tasks · D20–D27 · The control room

OBS-001 · OpenTelemetry instrumentation — traces and spans

Track OPS · Day D20–21 · Depends GW-002, FOUND-002

GOAL — One trace follows a request from the API edge through routing, retrieval, provider call and back.

SPEC - Otel SDK with OTLP export to the collector. Auto-instrument FastAPI, SQLAlchemy, Redis and httpx. - Manual spans for the stages that matter: `gateway.route`, `gateway.provider_call`, `knowledge.retrieve`, `knowledge.rerank`, `agent.step`, `eval.score`. - Propagate trace context into Celery tasks — a trace that ends at the queue boundary is half a trace and hides the slowest work. - Baggage carries `workspace_id` and `request_id` so every span can be filtered by tenant without re-joining tables. - Sampling: 100% of errors and slow requests, configurable head sampling for the rest, defaulting to 100% in development and 20% in production.

DONE WHEN — A single request produces one connected trace spanning API, database, provider and worker. Trace context survives the Celery boundary. Every span carries the workspace ID.

TABLES — traces, spans

OBS-002 · GenAI semantic conventions for LLM spans

Track OPS · Day D21–22 · Depends OBS-001

GOAL — LLM spans use standard attribute names, so any OTEL-compatible tool can read them without custom mapping.

SPEC - Follow the OpenTelemetry GenAI semantic conventions on provider-call spans: `gen_ai.system`, `gen_ai.request.model`, `gen_ai.response.model`, `gen_ai.request.temperature`, `gen_ai.request.max_tokens`, `gen_ai.usage.input_tokens`, `gen_ai.usage.output_tokens`, `gen_ai.response.finish_reasons`. - Platform-specific attributes namespaced separately under `nexus.*`: `nexus.workspace_id`, `nexus.policy_version`, `nexus.fallback_count`, `nexus.cost_usd`, `nexus.cached`. - Prompt and completion content on spans is opt-in per workspace and follows the same `logging_mode` as GW-008. Never emit content by default — trace backends are rarely governed as carefully as your database. - Span events for retries, fallbacks and circuit-breaker transitions, so a timeline explains what happened rather than just how long it took. - Retrieval spans carry `nexus.retrieval.chunks_returned`, `chunks_excluded_by_permission` and `top_score`.

DONE WHEN — Provider spans validate against the GenAI conventions. Cost and workspace are queryable as span attributes. Content is absent unless explicitly enabled. A fallback shows as a span event on the timeline.

TABLES — spans, span_events

OBS-003 · Trace persistence and query API

Track DATA · Day D22–23 · Depends OBS-001, INFRA-003

GOAL — Traces are queryable from the platform's own API, not only from an external tool.

SPEC - Collector exports to both the trace backend and a Postgres exporter writing `traces`, `spans` and `span_events`. - `traces` is partitioned by day; retention 30 days for full spans, and `metric_rollups` keeps the aggregates indefinitely. - `GET /v1/traces?workspace=&from=&to=&min_duration_ms=&status=&model=&has_fallback=` with cursor pagination. - `GET /v1/traces/{id}` returns the full span tree with timings, attributes and events, shaped for direct rendering as a timeline. - Correlate by `request_id` so a gateway request, its trace, its cost entry and its agent run all resolve from one identifier. Without that, debugging means four separate searches.

DONE WHEN — Traces persist to Postgres and survive a collector restart. Filtering by duration and model returns correct results. A single request ID resolves to its trace, cost entry and gateway request.

TABLES — traces, spans, span_events, gateway_requests, cost_entries

OBS-004 · Metric rollups — hourly and daily aggregation

Track DATA · Day D23–24 · Depends OBS-003, GW-006

GOAL — Dashboards read pre-aggregated rows, not raw request tables, so they stay fast as volume grows.

SPEC - Scheduled job aggregating into `metric_rollups` at hourly and daily grain, keyed by workspace, model, and where relevant user or agent. - Metrics: request count, success and error counts by error class, p50/p95/p99 latency, input and output tokens, cost, fallback count, cache hit rate, retrieval hit rate, tool-call success rate, human approval rate. - Percentiles from t-digest sketches stored per bucket, so they can be merged across buckets correctly. Averaging percentiles across time buckets produces a number that is simply wrong, and it is a mistake that survives a long time because it looks plausible. - Idempotent by `(period, grain, dimension_hash)` — a re-run overwrites rather than doubling. - Backfill command for a date range, so a job outage does not leave a permanent hole in the dashboards.

DONE WHEN — Rollups populate hourly and daily. Re-running produces identical numbers. p95 from merged sketches matches p95 computed from raw rows within tolerance. Backfill fills a deliberately skipped window.

TABLES — metric_rollups, cost_entries, gateway_requests, spans, agent_runs

OBS-005 · Prometheus metrics and Grafana dashboards

Track OPS · Day D24–25 · Depends OBS-001, FOUND-002

GOAL — Operational dashboards for the platform itself, separate from the product analytics in UI-003.

SPEC - `/metrics` endpoint exposing: `nexus_requests_total{workspace,model,status}`, `nexus_request_duration_seconds` histogram, `nexus_tokens_total{direction}`, `nexus_cost_usd_total`, `nexus_fallbacks_total{from,to,reason}`, `nexus_circuit_breaker_state{provider,model}`, `nexus_queue_depth{queue}`, `nexus_ingestion_jobs{status}`. - Keep cardinality controlled: workspace and model are acceptable label dimensions, user and request ID are not. - Three provisioned Grafana dashboards, committed as JSON in `/infra/grafana`: - **Platform health** — request rate, error rate, latency percentiles, queue depth - **Provider health** — per-provider latency and error rate, circuit breaker state, fallback flow - **Spend** — cost rate, spend by workspace and model, budget burn-down against the month - Dashboards are provisioned from source, not clicked together in the UI, so they survive a container rebuild.

DONE WHEN — Prometheus scrapes successfully. All three dashboards load with data from a seeded workload. A forced provider failure is visible on the provider dashboard within one scrape interval.

OBS-006 · Alert rules and failure budget tracking

Track OPS · Day D25–26 · Depends OBS-004

GOAL — The platform tells you it is degraded before a user does.

SPEC - `alert_rules` holds workspace-scoped rules: metric, comparison, threshold, evaluation window, consecutive-breach count and notification channel. - Seeded default rules: error rate above 5% over 5 minutes; p95 latency above 10 seconds over 5 minutes; any circuit breaker open; budget burn at 80% and 100%; ingestion job failure rate above 10%; retrieval returning zero results above 20% of queries. - Alerts fire once per breach with a resolution notification, not repeatedly. Require N consecutive breaches to suppress single-datapoint noise. - Channels: webhook, email, Slack-compatible incoming webhook. - Every alert links directly to the filtered trace query that shows the offending requests. An alert that does not lead you to the evidence just creates work.

DONE WHEN — A deliberately induced error rate fires exactly one alert and one resolution. Budget alerts fire at both thresholds. The alert payload contains a working link to the relevant traces.

TABLES — `alert_rules`, `metric_rollups`

UI-003 · Observability dashboard — cost, latency, usage, failures

Track UI · Day D25–27 · Depends OBS-004, UI-001

GOAL — The screen that answers “what is this costing us and what is going wrong”.

SPEC - **Header** — requests today, spend this month against budget as a burn-down, p95 latency, error rate, each with a sparkline and change against the prior period. - **Spend** — stacked area by model over time; tables by workspace, by user and by model with a token/cost/request breakdown; a “most expensive requests” list linking to traces. - **Performance** — latency percentiles by model, error breakdown by normalised error class, fallback flow showing which model fell back to which and how often. - **Usage** — requests by hour heatmap, top API keys, retrieval hit rate, tool-call success rate. - **Trace explorer** — filterable list, click through to the full span timeline with attributes and events. - Date range control on everything, defaulting to the last 7 days. Cost values render to four decimal places rather than rounding sub-cent amounts to zero. - Every chart is backed by `metric_rollups`, never by a live scan of `gateway_requests`.

DONE WHEN — All panels load under two seconds on a seeded month of data. Filters apply consistently across every panel. The trace explorer opens a full timeline. Budget burn-down matches the figure the gateway enforces.

Phase 4 — Evaluation Lab

6 tasks · D26–D33 · Where demos become engineering

EVAL-001 · Prompt registry with versioning

Track PLAT · Day D26–27 · Depends INFRA-003, AUTH-002

GOAL — Prompts are versioned artefacts with history, not string literals scattered through the codebase.

SPEC - `prompt_templates` is a named, workspace-scoped prompt. `prompt_versions` holds each immutable revision: content, variables schema, model defaults, changelog, author, created timestamp. - Variables declared as a JSON Schema; rendering validates inputs and fails loudly on a missing variable rather than emitting the literal placeholder into a prompt. - Labels point at versions: `production`, `staging`, `candidate`. Applications reference a label; promoting a version is a pointer move, and rollback is the same move backwards. - `GET /v1/prompts/{id}/diff?from=&to=` returns a rendered diff between versions. - Gateway requests record `prompt_version_id` when one was used, so a change in output quality can be traced to a prompt edit.

DONE WHEN — Editing a prompt creates a version rather than mutating one. A missing variable raises a clear error. Promoting and rolling back a label takes effect immediately for callers. The diff renders.

TABLES — `prompt_templates`, `prompt_versions`, `gateway_requests`

EVAL-002 · Evaluation datasets and item management

Track DATA · Day D27–28 · Depends EVAL-001

GOAL — Test sets that reflect what actually goes wrong in production.

SPEC - `eval_datasets` is a named, versioned collection of `eval_items`. Each item: input variables, expected output or expected behaviour, optional expected retrieval chunk IDs, tags and difficulty. - Three ways in, and the third matters most: - manual creation in the console - CSV/JSONL import - **promotion from production** — one click from a KB-008 feedback record or any logged gateway request, carrying the real input and the corrected expected output - Dataset splits (`train` / `dev` / `test`) so prompt iteration does not silently overfit the set used to report results. - Ship three seeded datasets: RAG factual accuracy, no-answer handling (questions the corpus deliberately cannot answer), and routing correctness (inputs whose correct destination model is known). - Datasets are versioned and immutable once an evaluation run has referenced them, so historical scores remain comparable.

DONE WHEN — A dataset imports from JSONL. A production feedback record promotes into an item with its real input attached. Modifying a referenced dataset creates a new version rather than editing in place. All three seeds load.

TABLES — `eval_datasets`, `eval_items`, `retrieval_queries`

EVAL-003 · Scorers — exact match, LLM-as-judge, retrieval metrics

Track DATA · Day D28–30 · Depends EVAL-002, GW-002

GOAL — Answer quality becomes a number you can defend.

SPEC - `Scorer` interface: `score(item, output, context) -> ScoreResult{value: float, passed: bool, reason: str}`. Every score carries a reason; a bare number tells you nothing when it drops. - Deterministic scorers: exact match, normalised match, regex, JSON schema validity, contains-all-keywords, latency threshold, cost threshold. - Retrieval scorers, computed against `expected_chunk_ids`: `precision@k`, `recall@k`, `MRR`, `NDCG`, and citation validity (do the cited chunks actually support the claim). - LLM-as-judge scorers: faithfulness (is the answer grounded in the retrieved context), relevance, completeness, and a pairwise preference comparator. Each runs through the gateway with a pinned judge model and a pinned judge prompt version — an unpinned judge makes historical scores meaningless. - Judge calibration: every judge scorer is validated against a small hand-labelled set, and its agreement rate with human labels is stored on the scorer. A judge you have not calibrated is a random number generator with good manners. - Scorers are composable — a run applies several and reports both individual and aggregate results.

DONE WHEN — Deterministic scorers return correct values on known cases. Retrieval metrics match a hand-computed example. The faithfulness judge correctly fails an answer containing an unsupported claim. Each judge's human-agreement rate is recorded.

TABLES — `scorers`, `eval_items`, `eval_results`, `gateway_requests`

EVAL-004 · Evaluation run executor with MLflow tracking

Track DATA · Day D30–31 · Depends EVAL-003, OBS-003

GOAL — Run a dataset against a configuration, score it and record the result as a comparable experiment.

SPEC - `POST /v1/eval/runs` body: `{ dataset_id, dataset_version, target: {type: prompt|rag|agent, ref, version}, model, scorer_ids, sample_size?, concurrency? }`. - Execution: bounded-concurrency async fan-out over items, per-item timeout, partial results preserved when a run is cancelled. - Every item execution goes through the gateway, so evaluation cost lands in `cost_entries` like everything else and the cost of an experiment is visible. - MLflow integration: one MLflow run per evaluation run, logging parameters (model, prompt version, retrieval config, scorer set), metrics (mean per scorer, pass rate, p95 latency, total cost) and artefacts (per-item results as JSONL, failure cases as a separate artefact). - `eval_results` stores per-item output, per-scorer score and reason, tokens, latency and the trace ID — so any individual failure opens straight into its trace. - Runs are reproducible: the run record pins dataset version, prompt version, model, scorer versions and judge prompt version. Re-running a pinned configuration reproduces the result within tolerance.

DONE WHEN — A run over 50 items completes with per-item results. MLflow shows the run with parameters, metrics and artefacts. Cancellation preserves completed items. A failing item opens into its trace. A pinned re-run reproduces the score.

EVAL-005 • Model and prompt comparison API

Track PLAT • Day D31-32 • Depends EVAL-004

GOAL — Answer the questions the platform exists to answer: which model, which prompt, which retrieval config.

SPEC - [POST /v1/eval/compare](#) takes 2-5 run IDs and returns per-scorer means with confidence intervals, per-item win/loss/tie, cost and latency per configuration, and the items where the configurations most disagree. - **Disagreement first.** The items where two configurations differ are where the information is; identical results are noise. Surface them at the top. - Report statistical significance with a paired bootstrap over per-item scores, and state plainly when a difference is not significant. A 2% improvement on 40 items is not a result, and the tool should say so rather than let a chart imply otherwise. - Cost-adjusted view: quality per dollar, which frequently changes the decision away from the strongest model. - [POST /v1/eval/sweep](#) runs one dataset against a matrix of models and prompt versions in a single call, producing a comparison automatically. - Comparison output exports as Markdown for pasting into a decision record.

DONE WHEN — Comparing two runs returns per-scorer deltas with intervals. A non-significant difference is labelled as such. The disagreement list surfaces genuinely different items. A sweep across three models and two prompts produces one comparison. Markdown export renders.

TABLES — eval_runs, eval_results, prompt_versions, models

UI-004 • Evaluation Lab UI — runs, comparisons, score breakdowns

Track UI • Day D31-33 • Depends EVAL-005, UI-001

GOAL — Configure a run, watch it, read the result and act on it.

SPEC - **Datasets** — list with item counts and splits; item editor; import; a queue of production items eligible for promotion. - **Prompts** — version list with diff view, label management, and a playground that runs a prompt against a few dataset items before committing to a full run. - **Runs** — configuration form, live progress with per-item completion, results table filterable to failures only, per-item drill-in showing input, output, every scorer's value and reason, and a link to the trace. - **Compare** — side-by-side columns, per-scorer bars with confidence intervals, a disagreement list as the default view rather than a buried tab, and cost-adjusted quality. - A run in progress shows current cost so an expensive sweep can be stopped before it finishes. - Failure triage is the primary flow: a run's default landing view is its failures, not its summary.

DONE WHEN — A run configures and executes from the UI with live progress. Failures filter in one click. Per-item drill-in shows scorer reasons and opens the trace. Comparison defaults to disagreements. Live cost displays during a run.

Phase 5 — Agent Studio

7 tasks · D32–D39 · Multi-step work under control

AGENT-001 · LangGraph runtime integration and checkpointing

Track PLAT · Day D32–34 · Depends GW-002, INFRA-003

GOAL — Agents run as durable graphs that survive a restart and can be resumed.

SPEC - LangGraph with a Postgres checkpointer, so state persists per thread and a crashed run resumes rather than restarting. - Every model call inside a graph goes through the gateway (GW-002), so agent runs inherit routing, cost tracking, quotas and policy without special-casing. - `agent_runs` is the top-level record: definition, version, trigger, input, status, token totals, cost, duration, step count. `agent_steps` records each node execution: type (`thought` | `tool_call` | `tool_result` | `approval` | `output` | `error`), inputs, outputs, tokens, duration. - Streaming: node-level events over SSE so a caller can watch progress rather than waiting on a black box. - Hard limits per run: max steps (default 25), max tool calls (default 15), max cost (default \$1.00), wall-clock timeout (default 300s). Breaching any of them terminates the run with the reason recorded. Unbounded agents are how a bug becomes an invoice. - Kill switch: `POST /v1/agents/runs/{id}/cancel` interrupts at the next checkpoint.

DONE WHEN — A multi-step agent completes with every step recorded. Killing the worker mid-run and restarting resumes from the last checkpoint. Exceeding the cost cap terminates with the reason on the run. Cancel takes effect within one step. Agent token usage appears in `cost_entries`.

TABLES — `agent_runs`, `agent_steps`, `gateway_requests`, `cost_entries`

AGENT-002 · Tool registry and permission grants

Track PLAT · Day D34–35 · Depends AGENT-001, AUTH-002

GOAL — Agents can only use tools they have been explicitly granted, with explicit limits.

SPEC - `tools` registry: name, description, JSON Schema for arguments, handler reference, side-effect class (`read` | `write` | `external` | `destructive`), and whether it requires approval by default. - Built-in tools: `knowledge_search` (KB-005, honouring the delegating principal's permissions), `sql_query` (read-only, allow-listed schemas, statement timeout), `http_request` (domain allow-list only), `create_ticket`, `send_notification`, `run_evaluation`. - `agent_tool_grants` binds a tool to an agent version with per-grant constraints: max calls per run, argument-level restrictions, and whether approval is required for this agent specifically. - Default deny. An agent with no grants has no tools. Adding a tool is a deliberate, audited act — every grant change writes to `audit_log`. - Sandboxing: tool handlers run with the delegating principal's permissions, never with elevated service credentials. An agent must not be a privilege-escalation path, and this is the single most likely security failure in a platform like this. - Every tool call is validated against its schema before execution and recorded in `agent_steps` with arguments and result.

DONE WHEN — An agent calling an ungranted tool fails with a clear error and an audit entry. Per-run call limits are enforced. `knowledge_search` returns only what the delegating user could retrieve directly. Every grant change is audited. Schema-invalid arguments are rejected before the handler runs.

TABLES — `tools`, `agent_tool_grants`, `agent_steps`, `audit_log`

AGENT-003 · Agent definition and versioning API

Track PLAT · Day D35–36 · Depends AGENT-002

GOAL — Agents are versioned configuration, deployable and rollback-able like prompts.

SPEC - `agent_definitions` — name, description, workspace, owner. `agent_versions` — graph definition, system prompt version reference, model or routing policy, tool grants, limits, changelog. - Graph defined declaratively as nodes and edges (JSON), so an agent can be built in the console without writing Python. Node types: `llm`, `tool`, `condition`, `approval`, `parallel`, `end`. - Labels as in EVAL-001: `production`, `staging`, `candidate`. Callers invoke by label. - Validation before save: no unreachable nodes, no cycles without an iteration bound, every referenced tool granted, every referenced prompt version existing. - `POST /v1/agents/{id}/test` runs a version against a sample input without promoting it. - Agent versions are evaluable — an agent version is a valid `target` for EVAL-004, so agent behaviour is measured the same way prompts are.

DONE WHEN — An agent is created, versioned and invoked by label. Validation rejects an unreachable node and an ungranted tool with specific messages. Rollback restores prior behaviour immediately. An agent version runs as an evaluation target.

TABLES — `agent_definitions`, `agent_versions`, `agent_tool_grants`, `prompt_versions`

AGENT-004 · Human-in-the-loop approval gates

Track PLAT · Day D35–36 · Depends AGENT-001, INFRA-003

GOAL — An agent pauses before consequential actions and waits for a person.

SPEC - An `approval` node interrupts the graph, persists the checkpoint and writes an `approval_requests` row with the proposed action, its arguments, the agent's reasoning and the full step history so far. - Approval is required when: the node is an explicit approval node, the tool's side-effect class is `write` or `destructive`, the grant sets `requires_approval`, or a governance policy (GOV-002) demands it. - `POST /v1/approvals/{id}` with `approve` | `reject` | `modify`. `modify` lets the reviewer edit the tool arguments before approval — most real reviews are corrections, not vetoes, and rejecting forces a full re-run. - Resume from the checkpoint on approval; on rejection the run ends with the reason in `agent_steps`. - Timeout per approval (default 24 hours) auto-rejecting with a timeout reason, so paused runs do not accumulate silently. - Approvals notify via the OBS-006 channels and appear in a console queue with a reviewer, decision and timestamp recorded for audit.

DONE WHEN — A destructive tool call pauses the run and creates an approval request with full context. Approving resumes from the checkpoint. Modify applies edited arguments. Rejection ends the run with a reason. Timeout auto-rejects. Every decision is audited with its reviewer.

AGENT-005 • Reference agent — document review with retrieval

Track PLAT • Day D36–37 • Depends AGENT-003, KB-006

GOAL — A working agent that proves the retrieval, gateway and orchestration layers compose. Read-only, so it is safe to demonstrate.

SPEC - Input: a document and a review checklist (for example a policy compliance checklist). - Graph: parse document → for each checklist item, `knowledge_search` against the policy collection → assess the document against the retrieved policy → collect findings → generate a structured report. - Tools: `knowledge_search` only. No writes, no external calls, no approval needed. - Output: findings as `{ checklist_item, status: compliant|non_compliant|unclear, evidence_from_document, policy_citation, confidence }`. - Every finding cites both the document passage and the policy chunk it was judged against. An unsupported finding is a defect. - `unclear` is a first-class outcome and the agent is instructed to use it rather than guess.

DONE WHEN — Running against a sample document produces findings for every checklist item. Each finding carries both citations. The full trace is visible in the console. Cost and tokens are recorded on the run.

TABLES — agent_runs, agent_steps, retrieval_queries, retrieval_citations

AGENT-006 • Reference agent — data quality triage with approval

Track DATA • Day D37–38 • Depends AGENT-004, AGENT-003

GOAL — The agent from the original brief: detect a pipeline failure, diagnose it, propose a fix and wait for a human before acting.

SPEC - Trigger: a webhook or scheduled check reporting a failed pipeline run. - Graph: fetch failure context → inspect recent logs (`sql_query` against a read-only ops schema) → compare current schema against the last known-good snapshot → classify the failure (`schema_drift` | `null_spike` | `volume_anomaly` | `upstream_timeout` | `unknown`) → draft a remediation → `approval gate` → on approval, `create_ticket` with the diagnosis and proposed fix. - Tools: `sql_query` (read-only), `knowledge_search` (runbooks collection), `create_ticket` (write, approval required). - The remediation draft must include the evidence it rests on: which query, which rows, which schema difference. A diagnosis without evidence cannot be reviewed, only trusted, and trusting an agent is exactly what this platform exists to avoid. - `unknown` is a valid classification that escalates to a human rather than inventing a cause. - Seed a reproducible failure scenario so the whole path is demonstrable end to end.

DONE WHEN — The seeded scenario runs, classifies correctly, pauses at approval with evidence attached, and creates a ticket only after approval. Rejection ends the run with no ticket. The full step trace including the approval decision renders in the console.

TABLES — agent_runs, agent_steps, approval_requests, tools

UI-005 • Agent Studio UI — builder, run traces, approval queue

Track UI • Day D37–39 • Depends AGENT-004, UI-001

GOAL — Build an agent, watch it work, approve its actions and understand its failures.

SPEC - **Agent list** — name, version, label, runs in the last 7 days, success rate, average cost, granted tools. - **Builder** — node-and-edge canvas for the graph, node configuration panels, tool grant management with the side-effect class shown prominently on each grant, limits editor, validation errors surfaced inline against the offending node. - **Run trace** — vertical timeline of steps: thoughts expandable, tool calls showing arguments and results, approval decisions with reviewer and timestamp, per-step tokens and cost, and total run cost in the header. - **Approval queue** — pending requests with the proposed action, the agent's reasoning, the preceding steps, and approve / reject / modify actions. Time remaining before auto-rejection shown on each item. - **Live run view** — streaming step events as the agent works. - Tools with `destructive` side effects render with a distinct treatment everywhere they appear. Someone granting a tool at speed should not be able to miss what it does.

DONE WHEN — An agent is built and saved from the canvas with validation errors shown inline. A live run streams steps. The approval queue supports all three decisions. The trace shows per-step cost. Destructive tools are visually unmistakable.

Phase 6 — Governance, Hardening & Release

7 tasks · D38–D45

GOV-001 · PII detection and redaction pipeline

Track PLAT · Day D38–39 · Depends GW-008, INFRA-003

GOAL — Sensitive data is detected before it is stored, and optionally before it reaches a provider.

SPEC - Detection with Presidio or an equivalent, extended with local patterns: national ID numbers, bank accounts, phone numbers in local formats, email, names, physical addresses, API keys and credentials. - Two enforcement points, configured per workspace: - **Pre-provider** — redact before the request leaves the platform. Strongest guarantee, and it changes the prompt, so it is off by default and must be an explicit decision. - **Pre-storage** — redact before writing to `gateway_messages`. On by default in `redacted` logging mode. - Detections recorded in `pii_detections` with entity type, character offsets, confidence and the action taken — never the detected value itself. Logging what you detected defeats the purpose. - `redaction_rules` allows per-workspace custom patterns and an allow-list for false positives (a product code that looks like an ID number). - Reversible redaction via a token vault for authorised roles only, so support can investigate with `governance.pii.reveal` while the default view stays redacted. - Detection runs asynchronously where it would otherwise add latency, except in pre-provider mode where it must be synchronous by definition.

DONE WHEN — A prompt containing an ID number and an email has both detected with correct offsets. Pre-storage redaction leaves no raw value in the database. Pre-provider redaction removes it from the outbound payload. Detections record type and offset but never the value. Allow-listed patterns are not flagged.

TABLES — `pii_detections`, `redaction_rules`, `gateway_messages`

GOV-002 · Policy engine — model allow-lists, tool blocks, data rules

Track PLAT · Day D39–40 · Depends GW-003, AGENT-002

GOAL — One policy layer governing which models, tools and data each workspace may use.

SPEC - `policies` are workspace-scoped, versioned, and evaluated at three enforcement points: gateway request, tool invocation, and retrieval. - Policy types: - **Model policy** — allowed and blocked models, maximum cost per request, whether external providers may receive data at all (some workloads must stay on self-hosted models) - **Tool policy** — blocked tools, tools requiring approval regardless of grant, argument-level constraints - **Data policy** — which collections may be sent to which providers, minimum sensitivity clearance, residency constraints - **Content policy** — blocked input patterns, required system prompt prefixes - Evaluation is deny-by-default when a policy matches and allow when none does. Every denial writes a `policy_violations` row with the policy, rule, principal, request and action taken. - Dry-run mode: a policy can be deployed in `monitor` mode, recording what it *would* have blocked without blocking. Never deploy a new policy straight to enforce on live traffic. - Policy evaluation is cached in Redis with a 60-second TTL and explicit invalidation on policy change; it sits in the hot path of every request.

DONE WHEN — A blocked model returns 403 naming the policy that denied it. A data policy prevents a restricted collection reaching an external provider. Monitor mode records violations without blocking. Every denial is recorded with its principal and rule. Policy changes take effect within 60 seconds.

TABLES — `policies`, `policy_violations`, `routing_policies`, `agent_tool_grants`, `collections`

GOV-003 · Immutable audit log and compliance export

Track DATA · Day D40–41 · Depends INFRA-003, AUTH-002

GOAL — A tamper-evident record of every consequential action, exportable for review.

SPEC - Every mutation to a governed table writes `audit_log`: table, record ID, operation, changed fields, old and new values, actor type, actor ID, workspace, request ID, IP and timestamp. - Also audit non-table events that matter: login, logout, failed authentication, API key creation and revocation, permission grants and revocations, policy changes, approval decisions, PII reveals, data exports. - Append-only enforced at the database level by the RLS policies from INFRA-003, not by application convention. Application-level immutability is a promise; database-level immutability is a control. - Tamper evidence: each row stores a hash chained to the previous row's hash within its partition. `GET /v1/audit/verify?from=&to=` re-walks the chain and reports any break. - `GET /v1/audit/export?from=&to=&format=json|csv` streams results, gated behind `governance.audit.read`. Exporting the audit log is itself audited. - Retention driven by `data_retention_schedules` with a documented legal basis per table; audit entries outlive the records they describe.

DONE WHEN — Every governed mutation appears with old and new values. `DELETE` and `UPDATE` on `audit_log` are rejected by the database. Chain verification passes on clean data and detects a manually altered row. Export streams a large range without exhausting memory. The export itself is audited.

TABLES — `audit_log`, `data_retention_schedules`, `policies`

GOV-004 · Security hardening

Track PLAT · Day D41–42 · Depends GW-007, KB-005, AGENT-002

GOAL — The platform survives review by someone whose job is finding the holes.

SPEC - **Input validation** — Pydantic models on every endpoint with `extra="forbid"`. Reject unexpected fields rather than ignoring them. Validate every UUID before it reaches a query. Bound every list and string length. - **Injection** — SQLAlchemy parameter binding everywhere; the `sql_query` tool runs against a read-only role with an allow-listed schema and a statement timeout, and its input is parsed and validated as a single SELECT rather than pattern-matched. - **Prompt injection** — untrusted content (retrieved chunks, tool results, uploaded documents) is delimited and labelled as untrusted in the prompt; tool-calling agents re-validate tool arguments after any step that consumed untrusted content. Treat this as risk reduction, not prevention — the durable control is that tools are permission-scoped and destructive actions need approval. - **Headers** — HSTS, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, a CSP without `unsafe-inline`, `Referrer-Policy: strict-origin-when-cross-origin`. - **CORS** — exact origin allow-list from configuration, never a wildcard with credentials. - **Secrets** — `gitLeaks` in CI, dependency audit (`pip-audit`, `pnpm audit`) on every PR, container image scan on build. - **SSRF** —

the `http_request` tool resolves and validates the destination IP against private ranges before connecting, and re-validates after redirects. - Write the negative tests: cross-workspace read attempts, ungranted tool invocation, a retrieval permission bypass attempt, a policy bypass attempt. They belong in CI, not in a document.

DONE WHEN — Unexpected fields return 400. Security headers present on every response. CORS blocks a disallowed origin. Secret scanning and dependency audit run in CI and fail the build on findings. The negative test suite passes and is part of the pipeline.

UI-006 · Governance Center and Admin Console

Track UI · Day D41–43 · Depends GOV-002, GOV-003, UI-001

GOAL — The screens an administrator and a compliance reviewer actually need.

SPEC - **Governance Center** - Policy list with type, mode (monitor/enforce), scope and violation count over the last 7 days - Policy editor with a dry-run preview showing what recent traffic the draft would have blocked - Violations feed: policy, principal, action, timestamp, link to the request - PII dashboard: detection counts by entity type and workspace over time, with values never displayed - Audit explorer with filters, chain verification status, and export - **Admin Console** - Workspaces: members, roles, budgets, logging mode, quotas - Users and API keys: creation, scope, last used, revocation - Provider and model configuration: credentials status (configured / not configured, never the value), health, pricing rates with effective dates - Feature flags with targeting - Destructive administrative actions require typed confirmation of the resource name. - The violations feed and audit explorer both link straight to the underlying request and trace. Governance screens that dead-end at a summary do not get used.

DONE WHEN — Policies are created and switched between monitor and enforce from the UI. Dry-run preview shows real would-block counts. The audit explorer filters and exports, and shows chain verification. PII counts display without values. Provider credential status shows configured state only.

SHIP-001 · Seed data, demo script and end-to-end walkthrough

Track PLAT · Day D43–44 · Depends UI-006, AGENT-006, EVAL-005, UI-003

GOAL — A fresh clone becomes a fully populated, demonstrable platform in one command.

SPEC - `make seed` creates: two workspaces with different policies and budgets, five users across all roles, a document collection of ~30 realistic documents, a populated knowledge index, 30 days of synthetic gateway traffic with realistic cost and latency distributions including deliberate failures and fallbacks, three evaluation datasets with completed runs, both reference agents with run history including one approved and one rejected approval, and a policy in each of monitor and enforce mode. - Synthetic traffic must be realistic enough that the dashboards look like a real system: a diurnal request pattern, a long tail of latency, a cost distribution skewed by a few expensive requests, and a visible incident window where a provider degraded. - Written demo script, 8 minutes, in order: route a cheap and an expensive request and show them landing on different models → force a provider failure and show the fallback → ask a knowledge question and open the retrieval inspector → compare two models on an evaluation dataset → run the data quality agent into its approval gate and approve it → open the observability dashboard on the cost of everything just done → show the audit trail of the session. - The demo ends on the audit log, because that is the argument the whole platform is making. - Full end-to-end test in CI running that same path headlessly.

DONE WHEN — `make seed` populates a fresh database and every dashboard shows plausible data. The demo script runs start to finish without a dead end. The end-to-end test passes in CI.

TABLES — all

SHIP-002 · Documentation, architecture diagram and write-up

Track OPS · Day D44–45 · Depends SHIP-001

GOAL — Someone technical can understand what you built, why, and what it cost, without you in the room.

SPEC - `README.md` : what the platform does, architecture diagram, quickstart to a running system in under 10 minutes, module tour with screenshots. - Architecture diagram showing request flow through gateway, policy, routing, provider, and the telemetry and cost paths — generated from source (Mermaid or D2) so it stays true. - `docs/decisions/` — one short ADR per significant choice: why RRF for hybrid retrieval, why effective-dated pricing, why database-level immutability for audit, why approval gates rather than tool restrictions alone, why the gateway sits in front of agent calls. - `docs/api/` — OpenAPI spec published, with a worked example per module. - Benchmarks with real numbers: retrieval latency at your corpus size, routing overhead added by the gateway in milliseconds, evaluation results comparing at least two models on one dataset, and cost per 1,000 requests under the default policy. - Portfolio write-up covering the problem, the design decisions and their trade-offs, what you would change, and what you learned. The trade-offs section is what distinguishes this from a tutorial follow-along, and it is what a hiring engineer reads first.

DONE WHEN — A fresh clone reaches a running system by following the README alone. The diagram matches the implementation. Every ADR states the alternative that was rejected and why. Benchmarks contain measured numbers, not estimates. The write-up names at least three trade-offs and their consequences.

Standing risks

Risk	Why it matters	Mitigation
Scope	Seven modules is a product, not a project. Half-finished modules read worse than three complete ones.	Phases 0–3 are the defensible core. If time runs out, ship Gateway + Knowledge Hub + Observability complete rather than all seven at 60%.
Provider cost during development	Evaluation sweeps and agent loops burn tokens fast, and a runaway loop is expensive before it is noticed.	<code>MockProvider</code> for all tests and CI. Per-run cost caps from day one (AGENT-001). Your own budget enforcement (GW-007) pointed at yourself.
pgvector at scale	HNSW index build time and memory grow sharply; a demo corpus hides this.	Benchmark at 100k chunks before committing to the parameters. Record the numbers in SHIP-002 — knowing the limit is more credible than not hitting it.
Heavily-tuned LLM indices	Evaluation sweeps that ask deep, validated, context-heavy questions	Used labeled QA from our index covered data the experiment

Uncalibrated LLM judges	Evaluation numbers that nobody has validated against human labels are worse than no numbers, because they get trusted.	Hand-label 50 items per judge scorer and store the agreement rate on the scorer (EVAL-003). Report it alongside every score.
Agent privilege escalation	An agent running with service credentials is a hole through every permission check in the platform.	Tools execute as the delegating principal (AGENT-002). Test it explicitly with a viewer-delegated agent.
Observability cost	Full-fidelity traces on every request can cost more to store than the inference they describe.	Sample below 100% outside development, keep raw spans 30 days, keep rollups forever (OBS-003, OBS-004).